



Finding XPath Bugs in XML Document Processors via Differential Testing

Shuxin Li*

shuxin.li.lv@gmail.com

Southern University of Science and Technology
China

Manuel Rigger

rigger@nus.edu.sg

National University of Singapore
Singapore

ABSTRACT

Extensible Markup Language (XML) is a widely used file format for data storage and transmission. Many XML processors support XPath, a query language that enables the extraction of elements from XML documents. These systems can be affected by logic bugs, which are bugs that cause the processor to return incorrect results. In order to tackle such bugs, we propose a new approach, which we realized as a system called XPress. As a test oracle, XPress relies on differential testing, which compares the results of multiple systems on the same test input, and identifies bugs through discrepancies in their outputs. As test inputs, XPress generates both XML documents and XPath queries. Aiming to generate meaningful queries that compute non-empty results, XPress selects a so-called targeted node to guide the XPath expression generation process. Using the targeted node, XPress generates XPath expressions that reference existing context related to the targeted node, such as its tag name and attributes, while also guaranteeing that a predicate evaluates to true before further expanding the query. We tested our approach on six mature XML processors, BaseX, eXist-DB, Saxon, PostgreSQL, libXML2, and a commercial database system. In total, we have found 27 unique bugs in these systems, of which 25 have been verified by the developers, and 20 of which have been fixed. XPress is efficient, as it finds 12 unique bugs in BaseX in 24 hours, which is $2\times$ as fast as naive random generation. We expect that the effectiveness and simplicity of our approach will help to improve the robustness of many XML processors.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging.**

KEYWORDS

XML processors, XPath generation, differential testing

ACM Reference Format:

Shuxin Li and Manuel Rigger. 2024. Finding XPath Bugs in XML Document Processors via Differential Testing. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*, April 14–20, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3597503.3639208>

*Work done during an internship at the National University of Singapore.



This work is licensed under a Creative Commons Attribution International 4.0 License. *ICSE '24*, April 14–20, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0217-4/ 24/04.

<https://doi.org/10.1145/3597503.3639208>

1 INTRODUCTION

Extensible Markup Language (XML) is a widely used file format for data storage and transmission. XPath is an expression language, which provides the ability to navigate through XML documents to select wanted nodes. XPath is also at the core of other XML query language standards such as XSLT [7] and XQuery [6], making it a fundamental XML query language.

Various XML document processors have been developed for extracting information from XML documents efficiently. We loosely categorize them depending on whether they can store XML documents in addition to processing them—that is, whether they are Database Management Systems (DBMSs), or provide only processing functionality. In terms of DBMSs specialized for XML, popular examples include BaseX [8] and eXist-DB [10]. Many general-purpose DBMSs such as Oracle Database [14], MySQL [13], and PostgreSQL [15] have adopted support for processing XML documents. In fact, out of the 10 most popular DBMSs according to the DB-engines ranking [9], 6 support at least partial XML document parsing. A popular example of a processor without storage functionality is Saxon. Saxon [18] is an in-memory XML processor that can be either used in a standalone way or embedded as a library. Finally, libxml2 [12] is a popular XML processing library written in C. XPath is supported by all of these processors.

XML document processors can be affected by logic bugs. Logic bugs are bugs that cause the XML processor to produce incorrect results without crashing the system, meaning that they can often go unnoticed. In order to find such bugs, developers mainly rely on test suites such as the XPathMark test suite for XPath [25], the W3C qt3 test suite [19], or hand-written unit tests. Manually writing tests requires much effort, and it is challenging to comprehensively cover the XML processors' functionality. To find logic bugs automatically, a so-called *test oracle* is required that can determine whether the system's output is correct in order to find logic bugs. Todic and Uzelac have proposed an automated testing technique for SQLServer's index support; their test oracle compared the results of a given query with and without index definition [41]. A limitation of this technique is that it is applicable only to finding index-related bugs in DBMSs. To the best of our knowledge, no other test oracles have been proposed in this context.

In order to detect XPath-related bugs in XML processors, we propose differential testing as an oracle. The core idea of differential testing is to use one input that is executed using multiple systems; any discrepancy in the results indicates a potential bug in the system. For testing XML processors, the input for the XML processors under test is an XML document and XPath expression, and the results are a sequence of XML nodes or values. Differential testing has been successfully applied in various related domains,

such as relational DBMSs [38], compilers [43, 45], JVM implementations [23], ORM systems [39], and graph DBMSs [44, 46]. Its effectiveness hinges on two main requirements. First, multiple systems to be compared must be available. As discussed above, various XML processors with XPath support exist. Second, for any valid input, the systems should produce the same result, since otherwise, a differential-testing approach raises many false alarms. This requirement is not always met, for example, when applying differential testing to relational DBMSs, where the “*common SQL subset is relatively small and changes with each release*” and NULL handling differs between DBMSs [38]. As we found, XPath is a well-defined language by the W3C standard, and XPath implementations of the same standard follow the same language rules, making differential testing highly applicable.

To generate test cases, we propose an approach that selects a so-called *targeted node* from the XML document, based on which we generate a query that is guaranteed to fetch at least that node. As such, it tackles two challenges that might prevent testing from exercising interesting behaviors. First, by generating the query based on the targeted node, we can guarantee that we access a tag name, attributes, and relative paths that exist with respect to at least the targeted node. Second, by rectifying predicates so that they evaluate to true for the targeted node, we can ensure that the result set is non-empty even for complex queries. A similar high-level idea has been proposed in the context of testing relational DBMSs, called *Pivoted Query Synthesis (PQS)* [36], where a pivot row was selected, based on which predicates were rectified to return true. Apart from applying that idea in a different context, we also propose a different rectification strategy that eschews mirroring the predicate’s execution logic in the testing tool, which was required for realizing PQS.

We implemented our approach as a tool named XPress,¹ which, to the best of our knowledge, is the first *general* automated testing tool for XML processors, and tested our method on six mature and widely-used XML processors BaseX, eXist-DB, Saxon, PostgreSQL, libXML2 and a commercial DBMS. The experimental results show that our approach is effective in detecting XPath-related logic bugs in XML processors. We found 27 previously unknown unique bugs, not covered by existing test suites, of which 19 were logic bugs. 25 of them have been confirmed, and 20 of them have been fixed. Furthermore, these test cases have been integrated into the aforementioned qt3 test suite, so that they can detect potential bugs in XML processors that we have not tested. Our experiments demonstrate that our proposed guided query generation process improves testing efficiency by finding 2× more unique bugs within 24 hours in BaseX as compared to random generation. Given the high effectiveness and efficiency of the approach, we believe it will likely be adopted by developers of XML processors to improve their systems.

To summarize, we make the following contributions:

- We propose the first general approach for automatically testing XML processors in order to find logic bugs.
- We implemented and evaluated our approach on six widely-used XML process systems, which successfully found 27 previously-unknown unique bugs.

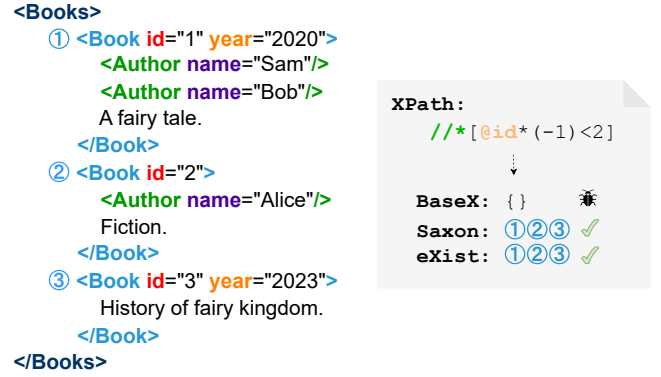


Figure 1: Example XML and motivating example.

2 BACKGROUND

Running example. Figure 1 shows a running example that we will subsequently use to explain basic XML and XPath concepts and outline the challenges of automated testing as applied in this context. The left shows an XML document with the root node `Books`, while the right shows an XPath expression `//*[ @id * (-1) < 2 ]`. We adapted this example from a bug-inducing test case that XPress discovered.² As shown, for the query on the document, BaseX returned an empty result, while both Saxon and eXist returned all three `Book` nodes.

XML. Extensible Markup Language (XML) is a text format for describing structured data. XML documents are trees that consist of nodes, as illustrated in Figure 1. An XML document has a single root element node (see `<Books>`). Each element node has a tag name (see `Books`, `Book`, and `Author`). Element nodes can include attribute nodes. For example, two of the `<Book>` nodes have both attribute nodes `id` and `year`. An element node can also include child element nodes; in the example, the `<Books>` node contains three child element nodes `<Book>`. Element nodes can hold *text contents*, which can be of any defined data type. For the `<Book>` node with attribute `id = 1`, the text content it holds is “A fairy tale”. Attribute nodes are disallowed from holding child nodes. In the example, `id` and `year` are integer-typed attribute nodes and the `name` attributes are string-typed attribute nodes.

XPath. The XPath language is an expression language that allows navigating the XML tree and hierarchic addressing of the element nodes. XPath is at the core of both *eXtensible Stylesheet Language - Transformation (XSLT)* [7] and *XQuery*, a more expressive query language for XML [6]. XSLT transforms XML documents into other formats and the XQuery language is a super-set of XPath expressions. XQuery extends XPath to provide functionalities such as node constructors and SQL-like clauses.

XPath structure. An XPath expression describes the selection and transformation of nodes of the XML tree. Figure 2 shows a simplified XPath 3.0 [4] grammar using EBNF notation from the W3C XML 1.0 standard [3]. We introduce the non-established terms *Section* and *Section Prefix* to describe our generation approach in

¹Our artifact is publicly available at <https://zenodo.org/records/10473926>

²<https://github.com/BaseXdb/baseX/issues/2188>

XPathExpression ::= Section+
SectionPrefix ::= ("|" | "/") AxisStep
AxisStep ::= Axis* NameTest
Section ::= SectionPrefix Predicate*

Figure 2: Simplified structure of XPath expressions.

Section 3.2. XPath expressions consist of one or more sections, and a section contains one section prefix followed by zero or more predicates. In Figure 1, the XPath expression `//*[@id*(-1)<2]` consists of a single section with section prefix `//*[@id*(-1)<2]` and a single predicate `[@id*(-1)<2]`. Each section prefix starts with either `|` or `/`. `|` is called the *path operator*, which accepts a node sequence as the left-hand operand and orders it in document order while eliminating duplicate nodes. `/` represents the abbreviated relative path `/descendant-or-self::node()`, which matches the current context and all descendant nodes of the current context, regardless of the intermediate path. An axis step consists of an optional axis and a name test.

XPath axes. Axes define the relationship between selected nodes and current context nodes. For example, the axis `parent::` selects all parent nodes of current context nodes. If omitted, it is equivalent to `child::`, which selects all direct children nodes of current context nodes. A name test is a string literal to fetch only nodes with the same tag name. It could also be a wildcard `*`, which matches all nodes without applying filters. The section prefix `//*[@id*(-1)<2]` in the example selects all descendant nodes of the document node, which is all element nodes in the document.

XPath predicates. Predicates in XPath include positional predicates and boolean predicates. Positional predicates contain an expression that evaluates to a single integer and select only values whose position in the context matches the integer value. In the XPath expression `/Books/Book[1]`, `[1]` is a positional predicate and selects only the first child of `<Books>`, which is the `<Book>` node with `@id=1`. Boolean predicates evaluate current context nodes to a boolean value according to a given expression and only nodes for which the predicate evaluates to `true` are selected. In Figure 1, `[@id * -1 < 2]` is a boolean predicate. The query `//*[@id * -1 < 2]` selects all nodes in the XML document with attribute `id` that satisfy `id * -1 < 2`. The three nodes with tag name `Book` in the document have attribute `id`, and all satisfy the condition. Therefore, if correctly evaluated, this query should return all three `Book` nodes.

Logic bug. For the test input in Figure 1, systems like Saxon and eXist-DB both returned a result set with three `Book` nodes, while BaseX returned an empty result set. The difference between the processors indicates a potential bug. Based on our manual analysis, we suspected that BaseX computed an incorrect result, which is why we reported it to the BaseX developers. They fixed the bug quickly. The reason for this bug was an incorrect simplification of the arithmetic expression $x * a > b$ to $x > b / a$. When the divisor is a negative number, the original operator `>` should be reversed to `<`.

XPath standard. There are majorly two different standards of XPath implementations in use today, which we need to consider in

our work. The XPath 1.0 standard was the first version. As a superset of XPath 1.0, the XPath 3.0 standard is the latest standard of the XPath language and provides more functionalities such as advanced data types and functions [5]. Most multi-model DBMSs, which support XPath queries, support only XPath 1.0 [1] (e.g., Oracle, MySQL, and PostgreSQL). While some specialized XML processors support also only XPath 1.0 (e.g., libXML2), others support the more recent XPath 3.0 standard (e.g., BaseX, eXist-DB, and Saxon).

XPath versions and differential testing. The same queries might produce different results under different standards. For example, for the XPath expression `Book/@name = false()`, under the XPath 1.0 standard, the expression is expected to return `true`. `@name` is first cast into its equivalent boolean value. In the current case `<Book>` has no `name` attribute, therefore, an empty node set is returned. The equivalent boolean value evaluates to `false` for empty nodes. Comparing `false` to `false` is equal, therefore `true` is returned. Under the XPath 3.0 standard, however, the result is expected to be `false`. `@name` returns an empty sequence and equality comparison between an empty sequence and a boolean value `false` would evaluate to `false`. Thus, applying differential testing to XML processors supporting different standards is infeasible.

3 APPROACH

Figure 3 shows an overview of the approach using the same example as in Figure 1. At a high level, our approach consists of three main steps. First, we randomly generate an XML document as the context for the following queries (step ①). We then generate an XPath expression that we will subsequently validate (step ② to step ⑤). Finally, we execute the XPath expression on the XML document using all engines under test and compare the resulting outputs to detect potential bugs (step ⑥). In the subsequent subsections, we explain these steps in reverse order to reflect their importance.

We guide the XPath expression generation towards queries that reference nodes and attributes present in the XML document and result in non-empty result sets based on the intuition that they are more likely to stress the underlying logic of the tested systems. To generate XPath expressions with non-empty result sets, we construct the query section-by-section and ensure that a non-empty result set is produced before proceeding with the next section. Each section consists of a section prefix and predicates, and we first generate the prefix (step ②) and then the predicate. By restricting the section prefix, we guarantee that the result contains at least one node. From the nodes selected by the section prefix, we randomly select a node as a target (step ③). We then generate a predicate aiming to select the targeted node using a bottom-up tree construction method (step ④). We rectify the predicate to ensure that the result set contains the targeted node (step ⑤). We repeat this process until the XPath query reaches the desired length.

3.1 Differential Testing for XML Processors

As detailed subsequently, differential testing enables us to find both logic bugs as well as internal errors when comparing the results of XML processors implementing the same XPath standard.

Query execution. When passing XML documents and XPath queries to different systems, we must account for the different

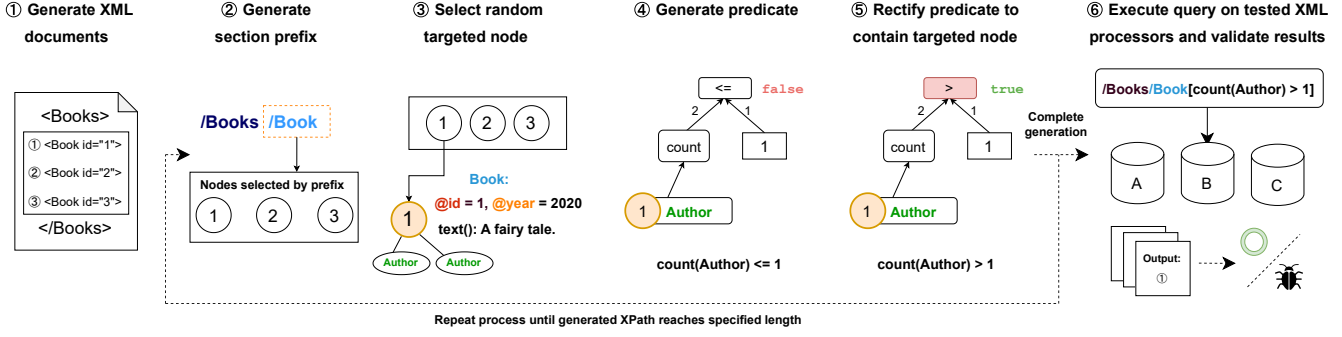


Figure 3: Overview of the approach implemented in XPress.

Listing 1: Execution of XPath using Oracle Database

```
CREATE TABLE t (a XMLType);
INSERT INTO t VALUES (XMLType(XML));
SELECT XMLQuery(XPATH PASSING a RETURNING CONTENT) FROM t;
```

Listing 2: Execution of XPath using Saxon in Java

```
XQueryExecutable exec = compiler.compile(XPATH);
XQueryEvaluator query = exec.load();
query.setContextItem(XML);
XdmValue result = query.evaluate();
```

input interfaces. For example, DBMSs use database connection interfaces to store and query data, while Saxon can be used as a library. To abstract this, we treat every XML processor implementation as a function that returns a result set and expects two string values, namely an XML document XML and an XPath query XPATH. Listing 1 shows an implementation of this interface for Oracle Database using SQL statements. It creates a table *t*, inserts the XML document—the *XMLType* constructor is used to convert the string to an XML data type—and uses an *XMLQuery* function call in a *SELECT* statement to compute the result set. For BaseX and eXist-db, similar to the commands shown for Oracle Database, we also start with an empty database and subsequently insert an XML document. Listing 2 shows an excerpt of the Java code for Saxon. First, the call to *compile* converts the textual XPath query to an executable object, which is then loaded. Unlike for the DBMSs, which require inserting data into a database, for Saxon, the XML document is simply associated with the query using the *setContextItem* call. The *evaluate* call computes the result, which is returned for comparison.

Bug identification. We identify both logic bugs and internal errors by comparing the returned results of different processors on the same XML document and query. We identify logic bugs when the tested systems return different node-set outputs for the same test cases. To parse and track the results easily under different output formats, we use unique node ids to identify element nodes. We detect internal errors as discrepancies with respect to errors. Rather than checking for an exact error message match, we validate whether all the systems produce an error, or all execute the XPath query successfully. If only a subset of the systems report an error for the same XPath query, we found a potential bug.

Different XPath standards. Our approach and tool are applicable to both XPath 1.0 and XPath 3.0. However, due to the differences in the formats, only processors using the same standard can be tested. Functionality that is supported only in XPath 3.0, can be disabled while generating test cases for processors that implement XPath 1.0. For example, sequence functions, such as *subsequence*, are defined only for the XPath 3.0 standard. When generating test cases for XPath 1.0 processors, we omit to generate *subsequence* function nodes for predicates. We did not encounter any functions or operators that were removed in the XPath 3.0 standard, so all expressions that we generate when testing XPath 1.0 processors can be used also when testing XPath 3.0 processors. By comparing only processors with the same XPath standard against each other, the difference in the results between different XPath standards (see Section 2) has no influence on the testing process.

3.2 XPath Expression Generation

In this section, we introduce how we generate XPath queries. We encountered two main challenges that we had to tackle when generating XPath expressions.

Non-existent elements. Randomly generated queries could be semantically correct, but reference non-existent nodes or attributes. For the document in Figure 1, *//Author[@id < 1]* is a valid XPath expression. However, none of the *Author* nodes contain an *id* attribute. Thus, XPath returns an empty sequence for each node, causing the predicate *@id < 1* to evaluate to false. We believe that queries, where only non-existent attributes or nodes are referenced, are less likely to exercise the logic of the processors under test, as subsequent operations are likely to evaluate to an empty sequence as well. Thus, we aim to avoid generating such queries.

Empty results. Randomly generated predicates might likely evaluate to false and cause queries to generate empty result sets. For the document in Figure 1, the XPath predicate *starts-with(text(), x)* identifies nodes whose text starts with *x*. If *x* is a randomly generated string, the possibility is high that no nodes in the current result set match the condition. Consequently, any use of the predicate would yield an empty result. Any subsequently added section would yield an empty result as well, meaning that such queries would be less likely to exercise the processor under test. Consequently, we want to avoid generating such predicates, in particular, when

they involve multiple sections. This relates to the first problem, as non-existent nodes or attributes can also introduce empty results.

Approach overview. We designed the XPath generation process of XPress tackling the two aforementioned issues. To create XPath expressions that refer to valid nodes and attributes to trigger deeper logic of the system under test, we generate queries that reference existent context relative to the so-called targeted node, such as its tag name and attributes (steps ③ and ④). Since randomly generated predicates might miss the targeted node from the result set, we rectify the generated expressions to ensure the inclusion of the targeted node (step ⑤).

Iterative section generation. We create XPath expressions section-by-section by executing step ② to step ⑤ for each section, which allows us to ensure non-empty results after generating each section. In the example, we first generate section `/Books` by selecting `<Books>` as the targeted node, and after executing steps ② to step ⑤, the result of `/Books` is non-empty—containing the node `<Books>`. Based on this, we further proceed to generate the next section `/Book[count(Author) > 1]` starting at step ②.

Section prefix. We randomly generate one of the applicable section prefixes. First, we randomly select the start of the section to be `/` or `//`. We then retrieve the current context node sequence by executing the expression `—/Books` in the example—on a processor. Based on the result, we include all possible axes that would not lead to an empty result set by simple conditional checks. We support all 11 axes described in the XPath 3.0 standard [4]. For example, applying the axis `/descendants::` will lead to a non-empty result, if at least one non-leaf node exists in the current selection. From the possible axes, we select a random one and apply it. When generating the section prefix `/Book`, the axis step is implicit. It is equivalent to `/Books/child::Book`, which selects all child nodes of the previously selected nodes. We again execute the query and retrieve the result node-set. We use the result for the name test, for which we either select a tag name from the result node-set, or use the wildcard `*`. By doing so, we are again guaranteed a non-empty result set. In the example, the tag name `Book` is selected and applied, resulting in the selection of all three `Book` nodes. In our artifact, we include a table that details the conditional checks for all 11 axes.

Target node selection. To generate targeted queries that fetch at least one node, we select a so-called *targeted node* to guide the predicate generation process. We use information about the target node, such as its text content, the attributes it holds, and its relationship to other nodes during the predicate generation. This is similar to the concept of the pivot row in PQS [36], which is a technique that has been proposed to test relational DBMSs. After the generated predicate is applied, we expect the target node to be included in the result node-set. In step ③, we select node 1 as the target node for the predicate generation process. Constraining the context to exist for the targeted node does not affect the evaluation of the expression on other candidate nodes and, therefore, still allows finding bugs that are triggered only when referring to nodes' non-existing attributes or child nodes.

Predicate generation. We use a tree structure to represent the predicate and take a bottom-up construction approach to enable

tracking of expression results along tree construction. We start generating the predicate from a specific subject, which is either the targeted node or a node sequence derived from the targeted node with equal probability. In the example, we select the `<Author>` child node sequence from the targeted node as the subject. We then iteratively apply random function nodes and supply function parameters to construct the predicate, until the predicate reaches a desired length. We keep track of the data type and value of the current sub-expression when constructing the predicate, by executing the sub-expression on one randomly chosen XML processor—we use this XML processor also for predicate rectification and we subsequently refer to this XML processor as the *designated* XML processor. We use the value and data type of the current sub-expression in the following two ways: by (1) selecting function nodes of according data types and (2) supplying arguments to reference existent context and triggering corner cases. Specifically, we select a random function node from functions that could accept the value of the current data type as input. A function node can either represent a function or an operation. In the example, `Author` is a node sequence and `count` is a randomly selected function from functions that accept node sequence as input. For function nodes that require additional arguments, we supply arguments while taking the current result value into consideration. As an example, when selecting attribute values from node sequences, we use name tests referencing existent attributes. For the `=` operator, we choose an operand that is equal to the current value with a high probability of triggering the equal case which is of low probability under random generation. Aside from constants, we also set the possibility for operands to be other predicate trees. Through this, we support the generation of expressions with multiple subject occurrences. Besides boolean predicates, we also apply positional predicates to the XPath expression randomly.

Predicate rectification. Lastly, we rectify the generated predicate to guarantee that the targeted node is contained in the final result set. We first execute the generated predicate on the designated XML processor. If the result set misses the targeted node, we rectify the predicate. To negate the predicate's result, we can always apply a `not` operator. However, as shown in Algorithm 1, we probabilistically apply more specific rectification for certain operators to uncover additional potential bugs. For logical operators such as `and`, both child expressions need to be modified to evaluate to true to contain the targeted node, while or needs only modification of one random child expression. For comparison operators, such as `<=`, we replace them with their opposite operators, which, in the example, is `>`. Thus, the targeted node is guaranteed to be contained in the result set.

3.3 XML Generation

In this section, we outline how we generate XML documents (step ①), which we do not consider part of our core contribution.

Tree creation. We use a bottom-up approach to generate XML documents. We first generate a number of node templates, which we use to generate XML nodes that have overlaps in terms of structure, as detailed below. We select one of these nodes as a root element. For the remaining nodes, we randomly assign each node to a parent. As XML documents support recursive structure, we allow cyclic

Algorithm 1 Predicate Rectification

```

1: function RECTIFYPREDICATE(predicate_node, targeted_node)
2:   c1 ← predicate_node.leftChild
3:   c2 ← predicate_node.rightChild
4:   if targeted_node in GETRESULT(predicate_node) then
5:     return
6:   if RANDOMPROB() < 0.5 then
7:     ADDNOT(predicate_node)
8:   return
9:   switch predicate_node do
10:    case or operator
11:      if RANDOMPROB() < 0.5 then
12:        RECTIFYPREDICATE(c1)
13:      else
14:        RECTIFYPREDICATE(c2)
15:    end case
16:    case and operator
17:      RECTIFYPREDICATE(c1)
18:      RECTIFYPREDICATE(c2)
19:    end case
20:    case comparison operator
21:      CHANGETOOPPOSITE(predicate_node)
22:    end case
23:    default:
24:      ADDNOT(predicate_node)
25:  end switch
26:  return

```

relationships. In Section 4, we provide details on how we configured the number of nodes in a document.

Node generation. We introduce how each element node is instantiated. By default, XML documents do not have to adhere to a specific schema, which is unlike, for example, relational DBMSs. Nevertheless, we want to generate element nodes that have overlaps in terms of structure, to test for more interesting behaviors. To that end, we generate element nodes based on so-called *node templates* that we randomly generate. A node template represents a type of node. For example, in Figure 1, *Book* is a node template whose tag name is *Book*, has attributes *id* and *year*, and has text content of string data type. To instantiate the template, we fill in values for the attributes and text contents. For each node we created in the aforementioned XML tree, we instantiate it with a randomly assigned template. In the example of Figure 1, we generated three nodes using the *Book* template. We assign random values for element nodes and their attributes according to the associated data types except *id*, to which we assign a unique identifier, which we use to unambiguously identify the processors' outputs (see Section 3.1). For the *<Book>* node with *id* = 1, we assign the random integer value 2020 to *year* and the random string value "A fairy tale" as its text content. Similar strategies have been applied also to other schema-less systems such as graph DBMSs [26, 29].

4 EVALUATION

In the evaluation, we sought to investigate whether our technique is effective and efficient in finding bugs for XPath expression processors. Specifically, we were interested in the following questions:

- Q1. Is XPress effective in finding new XPath-related bugs in established XML processors (see Section 4.1)?
- Q2. Does the query generation approach described in Section 3.2 improve the bug-finding efficiency of XPress with respect to real-world baselines and a random generation approach (see Section 4.2)?
- Q3. How does the differential testing test oracle compare to the state-of-the-art oracle (see Section 4.3)?
- Q4. What kind of XPath-related bugs might be overlooked by XPress (see Section 4.4)?

Tested XML Processors. We tested our method on six mature, well-known, and actively maintained XPath processors: BaseX, exist-DB, Saxon-HE, PostgreSQL, libXML2, and a commercial DBMS, whose name we have omitted due to its "DeWitt clause" [24]. We started testing on BaseX 10.4, eXist-DB 6.2.0, Saxon Home Edition 12.2, PostgreSQL version 15, and libXML2 commit version 106153. As bugs were resolved, we constantly updated to the latest available version. We selected BaseX, eXist-DB, and Saxon to be our main testing targets, because they all implement the more recent XPath 3.0 standard. BaseX ranks as the most popular Native XML DBMS on the DB-Engines Ranking [9]. eXist-DB is widely applied in data centers, systems, and platforms, as referenced on the eXist-DB reference page [11]. Saxon is an in-memory processor and therefore is not included in the DB-Engines rankings. However, the official website of Saxon [16] states: "More than 170 software vendors have built Saxon into their own applications" and "6 of the world's top 10 software vendors are Saxonica clients", demonstrating that Saxon is a widely-used and popular XML processor. For XPath 1.0 standard implementations, we tested PostgreSQL, libXML2, and the commercial DBMS. PostgreSQL is a popular open-source DBMS, which ranks 4 on the DB-Engines ranking and has 12.8k stars on GitHub. libXML2 is a software library developed for the GNOME project. The commercial DBMS is often considered the most popular and important DBMS overall, as also reflected in various rankings. All XML processors have been actively maintained for over 15 years.

Experimental setup. We implemented the tool, XPress, in around 8,000 LOC in Java. In our experiments, we configured it to generate XML documents that contain 1 to 50 nodes. We create half as many node templates as element nodes. For each XML document, we generated 200 XPath expressions. Each XPath expression had an equal possibility to hold 1 to 7 sections. We set one predicate to hold at most 10 subjects (see Section 3.2) and the depth of the predicate tree to be at most 10. We used the default settings of each XML processor. We conducted all our experiments using a personal computer with a 64-Core AMD EPYC 7763 CPU at 2.45GHz and 512GB memory running Ubuntu 22.04.

4.1 Effectiveness

In this section, we show XPress' effectiveness through the number of bugs found, developer feedback, and illustrative examples.

Table 1: Bugs found by XPress

XML Processor	Fixed	Confirmed	Reported	Total
BaseX	15	0	0	15
eXist-DB	1	5	0	6
Saxon	4	0	0	4
Commercial DBMS	0	0	2	2

Table 2: Category of Bugs found by XPress

XML Processor	Logic Bugs	Internal Errors
BaseX	10	5
eXist-DB	5	1
Saxon	2	2
Commercial DBMS	2	0

Methodology. We implemented the tool while intermittently testing the systems over a period of 3 months. For every found discrepancy, we reduced the test case. If the test case exhibited an unreported pattern, we considered it likely to be an unknown bug and reported it to the developers. Note that this was a best-effort approach, and that it is an unsolved problem of how to identify duplicate bugs effectively. Whether we considered a bug as unique was based on the developers’ verdict; we considered a bug only as unique if an issue was addressed through an independent bug fix. Unfixed bugs hinder testing, as the duplicates tend to be repeatedly triggered. To tackle this, we attempted to disable the construction of bug-inducing elements, and also ignored known discrepancy patterns before the reported bug was resolved.

Found bugs overview. As shown in Table 1, we successfully found 27 unique bugs in total, 15 in BaseX, 6 in eXist-DB, 4 in Saxon, and 2 in the commercial DBMS. As detailed subsequently, we could have reported additional bugs for eXist-DB and the commercial DBMS, but refrained from doing so due to the high number of unfixed bugs for eXist-DB, and lack of developer feedback for the commercial DBMS. The bug-inducing test cases we found were not covered by the W3C qt3 test suite [19], which contains around 30,000 tests for XPath and XQuery—Saxon 11.1 passes all applicable tests in the W3C qt3 test suite [17]. Out of the 27 bugs found, the majority, 19 bugs, were logic bugs. Based on developer feedback, we learned that among the 20 fixed bugs, at least 8 bugs were due to incorrect optimizations. We detected the remaining bugs through unexpected errors. All systems we tested were implemented in Java, so we did not observe any crash bugs. We did not find any bugs in PostgreSQL and libXML2, both of which are known to be robust systems. For example, previous bug-finding efforts on testing DBMSs using SQL queries also found no logic bug in PostgreSQL [34, 35].

Small-scope hypothesis. We observed that the reported bugs are mainly reproducible by short test cases. 70% of all the reported cases can be reproduced with an XML document that consists of only one node and 91% of XPath expression consists of only one section. The average length of the XML documents in the reported test cases was 12 characters and XPath expressions 30 characters. This

XML: `<T>1</T>` XPath: `//T[(@t >= 0) or (@t <= 1)]`
 Result: {}  | `<T>1</T>` 

Figure 4: Incorrect optimization of comparison conditions.

XML: `<S/>` XPath: `//S[last() * 150000 >= position()]`
 Result: `<S/>`  | {} 

Figure 5: Arithmetic overflow in pre-check conditions.

phenomenon is known as the small-scope hypothesis [21], and this observation has been exploited in testing work that systematically generates small test inputs [33].

Developer reception. Developer feedback is an important indicator of the bugs’ importance. A core developer of BaseX stated “Thanks for sharing the bug reports with us. I appreciate that, they’re definitely helpful.”³ All 15 bugs reported to BaseX were resolved within one month—10 bugs were resolved even within 24 hours. This indicates not only that the team was fast in resolving bugs, but also that the bug reports were considered valuable. Due to the timely fixes of the BaseX team, we invested most time and effort in testing BaseX. After encouragement from the developers of BaseX, we contributed the bug-inducing test cases to the W3C XQuery and XPath test suite [19]. Most bugs submitted to eXist-DB have not yet been fixed, which is likely the result of the many open issues (over 400). Nevertheless, the developers from eXist-DB confirmed the bugs quickly and also expressed appreciation towards the bug reports “thank you for finding and reporting.”⁴ Because the reported bugs remained unfixed for over two months, we stopped testing and reporting to eXist-DB after reporting the first few found inconsistencies due to the difficulties of filtering out duplicate bugs. We believe that XPress has the ability to find more bugs in eXist-DB after the known bugs are resolved. Similarly, for the commercial DBMS, since the developers did not follow up on the bugs that we reported, we stopped testing this DBMS. For Saxon, all four bugs reported were resolved quickly within one week’s time.

Selected bugs. Below, we give a few selected examples of bugs found by XPress to illustrate its bug-finding capability.

Incorrect optimization of comparison conditions. Figure 4 shows a fixed bug that we reported to BaseX.⁶ The XPath expression selects all T nodes with attribute @t that satisfies @t >= 0 or @t <= 1. When @t exists and is a numeric value, this is a condition that always evaluates to true. Therefore, an optimization in BaseX rewrote the predicate to true. However, when @t does not exist for node T, @t evaluates to an empty sequence and returns false for both @t >= 0 and @t <= 1. Before we reported this bug, this case was overlooked and resulted in an incorrect optimization.

³<https://www.mail-archive.com/basex-talk@mailman.uni-konstanz.de/msg15173.html>

⁴<https://www.mail-archive.com/basex-talk@mailman.uni-konstanz.de/msg15204.html>

⁵<https://github.com/eXist-db/exist/issues/4830>

⁶<https://github.com/BaseXdb/basex/issues/2190>

```

XPath:
tail(subsequence((1 to 2), 1, 2))
Result: "2" ✓ | {} ✗

```

Figure 6: Result of tail after subsequence off by one.

```

XML:      XPath:
<A><B></A> //*[((./.) /parent::* /last() ! (. > 1)) = true()]
Result: {} ✓ | <B/> ✗

```

Figure 7: Incorrect reduce in positional expressions.

Arithmetic overflow in pre-check conditions. Figure 5 shows a fixed bug that we reported to BaseX.⁷ `last()` and `position()` returns the context size and the context position from the dynamic context respectively. In the context XML document, the prefix `//S` selects only one node, and therefore both `last()` and `position()` return 1. Therefore, the condition is true and node S should be selected. In BaseX, an empty result set was returned. The problem was related to optimization for positional arguments in conditional comparisons. BaseX substituted `last()` with the greatest theoretical `last()` value and checked if the condition could evaluate to true. If not, the condition could not be satisfied regardless of the actual context and could be rewritten to false to reduce context analysis. When calculating the multiplication, as the theoretical maximum value for `last()` is a big integer, calculating the expression with `long` instead of `double` caused an overflow and produced the incorrect result.

Result of tail after subsequence off by one. Figure 6 shows a fixed bug that we reported to eXist-DB.⁸ `1 to 2` creates an integer sequence consisting of 1 and 2. The `subsequence()` function in this example selects two elements starting from index 1, and the `tail()` function returns a new sequence excluding the first element of the input sequence. The correct result is to return 2. Unexpectedly, eXist-DB returned an empty result set. This was caused by a mistake when processing a call to `tail` that has a call to `subsequence` as an argument, which incorrectly reduced the ending index by 1.

Incorrect reduce in positional expressions. Figure 7 shows a fixed bug that we reported to Saxon.⁹ The dot (.) stands for the current context in XPath expressions. For node B, `((./.) /parent::*` selects the single node A as the parent. Therefore, `last() = 1` and the condition evaluates to false. Saxon unexpectedly returned the node B. The `=` operator is considered to be an unordered operator, which does not require operands to be sorted. In Saxon, an optimization was applied to eschew removing duplicate nodes when evaluating the sub-expression, which resulted in A being selected twice and `last()` evaluated to 2. After we found and reported the bug, a patch was applied by the developers to remove the duplicates, when the left operand of `=` is positional sensitive.

4.2 Efficiency

Existing-generator baselines. We considered the only two—to the best of our knowledge—approaches to generate XPath expressions. Neither of them was specifically designed to be combined with a

XPath test oracle. XQgen [42] generates XPath queries for micro-benchmarking. Its generated predicates only check for sub-element existence. The XQuery generator designed by Todice and Uzelac [41] generates XPath queries for automatically testing index support in DBMSs. Given that indexes apply only to sargable queries (*i.e.*, simple comparisons), the expressions it generates are simple. Both approaches generate XPath expressions based on an XML schema, while XPress generates XPath expressions based on the actual XML document. Based on this, we expect both of them to have low applicability for our differential-testing approach. Given that neither implementations are publicly available, we re-implemented them based on the description in the papers.

Self-constructed baselines. We also constructed our own baselines to investigate the efficiency of the separate components of XPress. XPress has two main components, namely (1) the targeted predicate generation by using the targeted node to refer to existing nodes and attributes and (2) the predicate rectification to avoid empty result sets. To evaluate the effect of the components individually, we enabled them individually to test whether they improve XPress’s bug detection efficiency.

Configurations. We considered four configurations for our self-constructed baselines. Apart from our proposed approach introduced in Section 3.2 as (1) *Targeted*, we derive configuration (2) *Targeted without Rectification*, (3) *Untargeted with Rectification*, and (4) *Untargeted without Rectification*. In (2) *Targeted without Rectification*, we disable the rectification process, which would otherwise ensure targeted node selection. Since selecting a targeted node for predicate generation guidance always requires at least one node in the result set, we stop generating new sections after an empty result set is produced. In (3) *Untargeted with Rectification*, we generate predicates without using targeted node information to supply parameters that reference existent context and trigger corner cases for function nodes, while keeping the rectification to ensure that at least one node from the candidate set is included in the result set. In (4) *Untargeted without rectification*, we remove both components to generate predicates randomly, while omitting rectification.

Methodology. We set each baseline to run for 24 hours [30]. We repeated each experiment 10 times to account for potential performance deviations, and report the arithmetic mean for all metrics. As our testing target, we selected BaseX 10.4, which is the BaseX version that we first started testing. The reason for selecting BaseX as a representative is that we found most bugs in BaseX and all bugs were fixed, allowing us to determine the number of *unique* bugs we found in a testing campaign by deduplicating bug-inducing test cases automatically. Specifically, given two bug-inducing test cases, we could determine whether they trigger the same underlying bug by identifying their fix commits; only if their associated fix commit are different, do we consider the bugs unique. This is a best-effort technique, as, for example, one fix commit might address multiple bugs. We disabled the generation of the `has-children` functions as well as using relative XPath expressions in predicates, as they consistently lead to crashes, triggering known bugs.

Results of existing generators. Neither XQGen nor the Combined XML/XQuery generator found bugs in our experiment. This is expected, as previously proposed approaches were not designed for

⁷<https://github.com/BaseXdb/basex/issues/2220>

⁸<https://github.com/eXist-db/exist/issues/4830>

⁹<https://saxonica.plan.io/issues/6093?pn=1#change-24136>

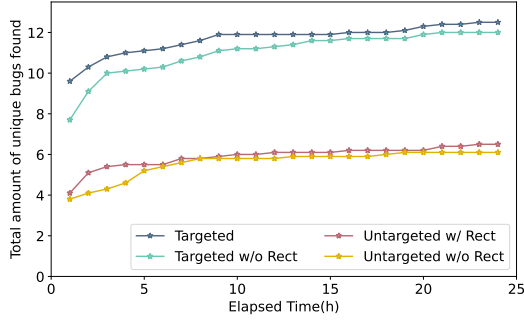


Figure 8: Average number of unique bugs found under different configurations in 24 hours across 10 runs.

Table 3: Average bug report collection under different configurations in 24 hours across 10 runs.

Config	Total cases	Differences detected	Unique bugs	Non-empty result
Targeted	6.6M	11.8K	12.5	100%
Targeted w/o Rect	9.4M	10.2K	12	66%
Untargeted w/ Rect	8.8M	1.4K	6.5	100%
Untargeted w/o Rect	13.5M	0.6K	6.1	44%

automated testing. As mentioned above, XQGen generates predicates that only check for element existence. The XQuery generator designed by Todic and Uzelac generates simple predicates that include at most one comparison operator.

Results of different configurations. As Figure 8 shows, our proposed approach, *Targeted* outperforms the other configurations. Within 24 hours, it found the most number of unique bugs (namely 12.5). Both configurations with targeted generation clearly outperformed the untargeted approaches, while rectification shows a similar performance in the speed of bug detection. As shown in Table 3, both targeted generation and rectification reduce the testing throughput, as they obtain intermediate results using the XML processor under test. Despite generating only 50% of the number of test cases as compared to (4) *Untargeted without Rectification*, (1) *Targeted* detected 20× more bug-inducing test cases and 2× more unique bugs. The results show that selecting a target node to guide the XPath generation process improves testing efficiency significantly. As observed above when discussing the small-scope hypothesis, most of the bugs that we found can be reproduced using a single section, explaining the limited effectiveness of rectification. However, we still believe that rectification is an important component, since without it, bugs requiring multiple sections with non-empty results could hardly be found.

Code coverage. We collected code coverage for three processors’ core modules for XPress for 24 hours [30] of execution. The result is shown in Table 4. To put the numbers in relation, we collected coverage also for the projects’ test suites; Saxon has no publicly available test suites and is therefore excluded. For the three XML processors, the line coverage ranged from 15% to 20%, and the

Table 4: Code coverage of tested systems in 24 Hours

Approach	BaseX		eXist		Saxon	
	Line	Branch	Line	Branch	Line	Branch
XPress	20%	16%	18%	10%	15%	10%
Unit Tests	67%	58%	52%	47%	-	-

branch coverage ranged from 10% to 16%. The coverage percentages are low, which is expected. The main reason for low code coverage is that XML processors typically also have other components than XPath processing. Taking BaseX as an example, around 21% of uncovered code was GUI-related, 10% was due to lack of full-text functionality support, and 5% were database commands. In Saxon, as another example, XSLT modules have not been covered. A further 18% uncovered code in BaseX involved unimplemented functions; it would be straightforward to implement many additional ones, such as math functions, but the many functions available would make this a tedious task. In Section 4.4, we detail unsupported XPath features, implementing which might allow us to find more bugs. XPress’s test-case generation process primarily aims at generating semantically valid expressions, which results in low error-checking branch coverage, quantifying which is difficult, as the relevant code is spread throughout the code base.

4.3 Comparison to the State of the Art

We are aware of only one automated testing approach that has been proposed to test XML processors [41]. It tackled the test oracle problem by using differential testing by comparing the results of Microsoft’s SQLServer with and without using indexes. Their approach was specifically designed to test SQLServer’s index support and is not publicly available. Due to the narrow testing scope, and since the tool is not publicly available, we could not conduct experiments to directly compare the approaches. However, we further extended our tool to support differential testing with index configurations. Both approaches are complementary, as XPress could not only use differential testing among various XML processors, but also create or omit indexes to find additional bugs.

Index support in BaseX, eXist-DB, Saxon, and libxml2. Database indexes are data structures built to speed up data retrieval [31] and are DBMS-specific. Not all XML processors are DBMSs—as in-memory processors, Saxon and libxml2 lack support for indexes. BaseX and eXist-DB both enable structural indexes, such as storing all distinct paths of nodes by default. For value indexes to optimize querying on content values, BaseX creates text index and attribute index automatically. Users can further define additional indexes. Additionally, BaseX provides token indexes, which apply to specific functions, such as `contains-token`. eXist supports range indexes, which could be defined for specific nodes or attributes to speed up related comparison searches on their contents.

Methodology. We tested eXist’s range index and BaseX’s token index using the XPath expression generation approach as described in Section 3.2. Due to the found unfixed bugs in eXist, we conducted differential testing within eXist by checking the results with and

```

XML:      XPath:
<M v="a"/> //M/descendant-or-self::M[contains-token(@v, "a")]
Result: <M/> ✓ | {} (create index token) ✗

```

Figure 9: Found bug with token index in BaseX.

without range index definition. For BaseX, we defined a token index and compared its results directly with the results of Saxon.

Results. Throughout the testing method, we detected one additional bug for BaseX¹⁰ and found no additional bugs in eXist. We reported the found bug shown in Figure 9 to the BaseX developers, who quickly fixed it. The query selects all nodes with tag name M in the document which holds attribute v that contains token "a". BaseX returned node M without token index, as expected, while unexpectedly returning an empty result set when not using an index. Overall, while the results suggest that using or removing indexes might find additional bugs, doing so had low effectiveness. A potential explanation could be that our test-case generation approach does not consider when indexes could be applied, which might result in low testing efficiency.

4.4 Analysis of BaseX Historical Bug Reports

Unlike formal verification approaches, automatic testing approaches might miss bugs in the system tested. Due to the lack of ground truth, we cannot generally determine which bugs are overlooked by our approach. However, as a best-effort approach, we studied historical bug reports in order to determine whether XPress could have found them.

Bug reports. We analyzed all historical BaseX bug reports in its GitHub bug tracker. We selected BaseX, because the majority of issues are closed (1618 out of 1640). The issue tracker of BaseX is used for confirmed bug reports filtered from reports from the mailing list, and the BaseX maintainers carefully label and document them. For these reasons, it was easy to identify and classify the underlying problem of each bug report.

Methodology. We manually analyzed all historical bug issues until 2023 Apr 17 in BaseX, which were 1597 issues, after excluding the issues we reported. To confine the study of bug reports within the scope of XPath, we selected bug reports triggered by only XPath expressions. To determine whether a bug could be theoretically found by XPress, we mainly checked three aspects of the reports. For XPress to cover the test case, both the XML document and the XPath expression in the test case should not include any unimplemented functions or language features. Second, we could construct the sections and the predicate tree structure of XPress for involved predicates to form the pattern of the bug-inducing XPath expression. Third, XML processors should disagree on the result set. Note that this is a best-effort approach, because we might both incorrectly conclude that XPress might find a bug (e.g., it might be unlikely that the test case would be generated in practice) or incorrectly conclude that a bug cannot be found even when a different test-case within the reach of XPress would trigger the same underlying bug.

Results. Out of the total 78 bugs that we collected, we identified 20 bugs that could have been detected by XPress. For the other 58 bugs, we identified 4 kinds of bugs that XPress would have failed to find, namely due to (1) unimplemented functionalities (51 cases), (2) invalid inputs where the expected result would be an error (6 cases), (3) processors producing different results (2 cases), and (4) miscellaneous other issues (6 cases). Bugs belonging to more than one group are included in all involved groups. The differential testing oracle fails to detect the bugs with processors producing different results, while we consider the other categories mostly as implementation limitations in test-case generation. Therefore, out of all 78 bugs, 76 bugs (97%) could be detected through differential testing. This further demonstrates the effectiveness of employing a differential testing oracle for XPath-related testing.

Unimplemented functionalities. Most uncovered bug reports are due to unimplemented functionalities. Unsupported functions include constructors defined by the XML or XPath language standards, array and map functions, and also constructors of derived datatypes [2], such as `xs:NMTokens`. Given enough time, it would be straightforward to implement them in XPress. For/while loops, variable declaration, if-else conditional expressions, and self-defined functions are also unimplemented. These could be supported based on approaches that have been proposed in the context of compiler testing [32, 43]. Neither the XML documents nor XPath expressions that XPress constructs involve namespaces, which allow distinguishing items with the same tag name. They could be integrated into the XPress test-case generator. By implementing all these features, an additional 38 bugs (48%) could have been found.

Expected errors. Bug reports grouped into *expected is error* refers to invalid test cases, which are successfully executed instead of throwing an error. XPress constructs both syntactically and semantically valid expressions and therefore could not detect bugs within this category. However, the differential testing oracle could detect these bugs by comparing the errors of the different XML processors.

Different results. The different result category contains queries for which different processors intentionally produce different results, which shows the limitation of the differential testing oracle. One example is the function `id`, which selects nodes with `xml:id` attributes. BaseX takes attributes named as `id` as `xml:id` attributes, while Saxon and eXist-DB require an explicit declaration.

5 RELATED WORK

While various related approaches to our work exist, to the best of our knowledge, we propose the first general approach to testing XML processors to find logic bugs. As discussed above, the most closely related work proposed testing the index support of SQLServer in the context of XPath and XQuery [41], which, to the best of our knowledge, is the only work that has tackled the test-oracle problem for XML processors, but is limited in scope.

Testing XPath functionality. Various approaches to benchmarking XPath implementations or test suites for them have been proposed, the most representative being XPathMark and the W3C qt3 test suite. XPathMark [25] is a benchmark for testing XML processors' XPath standard 1.0 functionality, containing both correctness

¹⁰<https://github.com/BaseXdb/basex/issues/2222>

as well as performance tests. The W3C qt3 test suite developed by the W3C XQuery and XSLT Working Groups [19] contains around 30,000 tests for XPath and XQuery targeting XPath 3.0 and later versions, which cover a broad range of functions and expressions.

XML-related automated synthetic data generation. Previous works have proposed approaches for automatically generating XML-related data, such as XML documents, XPath, and XQuery expressions. Aboulmaga et al. proposed an XML document generator to generate synthetic, but complex, structured XML data by introducing recursion and repetition on tag name assignment and controlling the element frequency distribution [20]. Rychnovský and Holubová proposed an approach to generate XML documents related to given XPath queries from a specific XML schema to improve query efficiency [37], which is useful for developers to create micro-benchmarks for testing performance over certain XPath expressions. XQGen [42] is a tool for generating XPath queries that conform to a given XML schema, allowing users to specify multiple parameters, such as the percentage of empty queries desired and the percentage of queries with predicates. XPath generated by XQGen includes only direct node tests without introducing complex expressions, such as axes or function transformations. Similarly, the XQuery generator designed by Todici and Uzelac [41] includes XQuery FLWOR expressions, but the logic predicate consists only of simple operations, such as value comparisons. Neither of these works tackled the test oracle problem, and, as indicated by the results in Section 4.3, given their different focus, they cannot be effectively combined with a differential testing oracle.

Targeted test case generation. Many testing tools guide their test case generation process to improve testing efficiency, for random approaches such as random byte mutation used in fuzzing approaches generate a large proportion of invalid queries [47]. DynSQL [27] guides the fuzzing process of DBMSs towards increased code coverage and high statement validity. APOLLO [28] is a system for detecting performance regression bugs in DBMSs. It increases the probability of including components from previously encountered performance issues. Cynthia [39] was proposed to test Object Relational Mappers (ORMs) and generates targeted databases dependent on generated abstract SQL queries, which are likely to return non-empty results. Query Plan Guidance (QPG) [22] guides testing towards exploring more unique query plans.

Pivoted Query Synthesis. The *targeted node* in XPress was inspired by the *pivot row* in *Pivoted Query Synthesis (PQS)* [36], which was originally proposed to test relational DBMSs. PQS' and XPress' commonality is that they select a random element, in PQS, a row in the database, while for XPress, a node in an XML document, based on which they generate a query that is guaranteed to fetch the element. However, both the purpose and use of the targeted node and pivot row differ. In PQS, the pivot row is used both for test-case generation and to construct the test oracle, by evaluating an expression and ensuring that it evaluates to true for the pivot row so that it can be used in a query that is guaranteed to fetch the row. Doing so requires a naive reimplementations of all the DBMSs' operators that should be tested, which incurs a high implementation effort, as highlighted in follow-up work [?]. In XPress, the targeted node is used only for test-case generation, to improve

testing efficiency and to ensure non-empty intermediate results; to this end, XPress uses the XML processor to determine the result of the expression, rather than requiring the reimplementations of operators. In addition, for predicate rectification, XPress provides operator-specific rules, rather than relying on a generic one, aiming to generate more interesting test cases. The high-level idea of a pivot element also inspired other works; for example, recent work on Android testing introduced the concept of a *pivot layout* [40].

6 CONCLUSION

This paper has presented a general automated testing approach for detecting XPath-related logic bugs in XML processors. We demonstrate that differential testing is applicable in this domain, since XML processors widely adhere to the XPath standards. To generate interesting XPath queries, our approach selects a so-called targeted node to guide predicate generation and predicate rectification to ensure the inclusion of that node. Our evaluation shows that this improves the number of bugs detected in 24 hours to 2× as compared to random generation. More importantly, we have successfully detected 27 previously unknown, unique bugs in six mature XML processing systems. We believe that this high number is surprising, given that XML processors are an essential part of our computing infrastructure, with the first XPath standard having been proposed more than 20 years ago, and the systems that we have tested having been maintained for at least 15 years. We believe that XPress, given its simplicity and generality, has a high chance of being integrated into the toolbox of XML processor developers. Furthermore, we believe that our work might inspire testing approaches for other XML standards, such as XQuery or XSLT.

ACKNOWLEDGMENTS

This research was supported by a Ministry of Education (MOE) Academic Research Fund (AcRF) Tier 1 grant.

REFERENCES

- [1] 1999. *XML Path Language (XPath) Version 1.0 W3C Recommendation*. Retrieved July 17, 2023 from <https://www.w3.org/TR/1999/REC-xpath-19991116/>
- [2] 2004. *XML Schema Part 2: Datatypes Second Edition - Built-in datatypes*. Retrieved July 17, 2023 from <https://www.w3.org/TR/xmlschema-2/#built-in-datatypes>
- [3] 2008. *EBNF notation from the W3C Extensible Markup Language (XML) 1.0 (Fifth Edition)*. Retrieved July 17, 2023 from <https://www.w3.org/TR/REC-xml/>
- [4] 2014. *XML Path Language (XPath) 3.0 W3C Recommendation*. Retrieved July 17, 2023 from <https://www.w3.org/TR/xpath-30/>
- [5] 2014. *XPath and XQuery Functions and Operators 3.0 W3C Recommendation*. Retrieved July 17, 2023 from <https://www.w3.org/TR/xpath-functions-30/>
- [6] 2017. *XQuery 3.1: An XML Query Language W3C Recommendation*. Retrieved July 17, 2023 from <https://www.w3.org/TR/xquery-31/>
- [7] 2017. *XSL Transformations (XSLT) Version 3.0 W3C Recommendation*. Retrieved July 17, 2023 from <https://www.w3.org/TR/xslt-30/>
- [8] 2023. *BaseX*. Retrieved July 31, 2023 from <https://basex.org/>
- [9] 2023. *DB-Engines Ranking*. Retrieved July 6, 2023 from <https://db-engines.com/en/ranking>
- [10] 2023. *eXist-DB*. Retrieved July 31, 2023 from <http://exist-db.org/exist/apps/homepage/index.html>
- [11] 2023. *eXist DB reference page*. Retrieved July 6, 2023 from <http://exist-db.org/exist/apps/homepage/references.html>
- [12] 2023. *libXML2*. Retrieved July 31, 2023 from <https://gitlab.gnome.org/GNOME/libxml2>
- [13] 2023. *MySQL*. Retrieved July 31, 2023 from <https://www.mysql.com/>
- [14] 2023. *Oracle Database*. Retrieved July 31, 2023 from <https://www.oracle.com/database/>
- [15] 2023. *PostgreSQL*. Retrieved July 31, 2023 from <https://www.postgresql.org/>
- [16] 2023. *Saxon home page*. Retrieved July 6, 2023 from <https://saxonica.com/html/welcome/welcome.html>

- [17] 2023. *Saxon XQuery 3.1 conformance page*. Retrieved July 13, 2023 from <https://www.saxonica.com/documentation12/#!conformance/xquery31>
- [18] 2023. *Saxonica*. Retrieved July 31, 2023 from <https://saxonica.com/>
- [19] 2023. *W3C qt3 test suite github repository*. Retrieved July 11, 2023 from <https://github.com/w3c/qt3tests>
- [20] Jeffrey F. Naughton Aboulmaga, Ashraf and Chun Zhang. 2001. Generating Synthetic Complex-Structured XML Data. *WebDB*. 1 (2001), 79–84.
- [21] Alexandr Andoni, Dumitru Daniliuc, Sarfraz Khurshid, and Darko Marinov. 2003. Evaluating the “small scope hypothesis”.
- [22] Jinsheng Ba and Manuel Rigger. 2023. Testing Database Engines via Query Plan Guidance. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 2060–2071. <https://doi.org/10.1109/ICSE48619.2023.00174>
- [23] Yuting Chen, Ting Su, Chengnian Sun, Zhendong Su, and Jianjun Zhao. 2016. Coverage-Directed Differential Testing of JVM Implementations. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Santa Barbara, CA, USA) (PLDI '16). Association for Computing Machinery, New York, NY, USA, 85–99. <https://doi.org/10.1145/2908080.2908095>
- [24] Timothy Dyck. 2002. *DB Test Pioneer Makes History*. Retrieved July 31, 2023 from <https://www.eweek.com/development/db-test-pioneer-makes-history/>
- [25] Massimo Franceschet. 2005. XPathMark: An XPath Benchmark for the XMark Generated Data. In *Database and XML Technologies*, Stéphane Bressan, Stefano Ceri, Ela Hunt, Zachary G. Ives, Zohra Bellahsene, Michael Rys, and Rainer Unland (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 129–143.
- [26] Ziyue Hua, Wei Lin, Luyao Ren, Zongyang Li, Lu Zhang, Wenpin Jiao, and Tao Xie. 2023. GDsmith: Detecting Bugs in Cypher Graph Database Engines. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3597926.3598046>
- [27] Zu-Ming Jiang, Jia-Ju Bai, and Zhendong Su. 2023. DynSQL: Stateful Fuzzing for Database Management Systems with Complex and Valid SQL Query Generation. In *Proceedings of the 32nd USENIX Conference on Security Symposium* (Anaheim, CA, USA) (SEC '23). USENIX Association, USA, Article 277, 17 pages.
- [28] Jinho Jung, Hong Hu, Joy Arulraj, Taesoo Kim, and Woonhak Kang. 2019. APOLLO: Automatic Detection and Diagnosis of Performance Regressions in Database Systems. *Proc. VLDB Endow.* 13, 1 (sep 2019), 57–70. <https://doi.org/10.14778/3357377.3357382>
- [29] Matteo Kamm, Manuel Rigger, Chengyu Zhang, and Zhendong Su. 2023. Testing Graph Database Engines via Query Partitioning. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3597926.3598044>
- [30] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. 2018. Evaluating Fuzz Testing. *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security* (2018). <https://doi.org/10.1145/3243734.3243804>
- [31] Quanzhong Li and Bongki Moon. 2001. Indexing and Querying XML Data for Regular Path Expressions. In *Proceedings of the 27th International Conference on Very Large Data Bases (VLDB '01)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 361–370.
- [32] Vsevolod Livinskii, Dmitry Babokin, and John Regehr. 2023. Fuzzing Loop Optimizations in Compilers for C++ and Data-Parallel Languages. *Proc. ACM Program. Lang.* 7, PLDI, Article 181 (jun 2023), 22 pages. <https://doi.org/10.1145/3591295>
- [33] Jayashree Mohan, Ashlie Martinez, Soujanya Ponnappalli, Pandian Raju, and Vijay Chidambaram. 2018. Finding Crash-Consistency Bugs with Bounded Black-Box Crash Testing. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 33–50.
- [34] Manuel Rigger and Zhendong Su. 2020. Detecting Optimization Bugs in Database Engines via Non-Optimizing Reference Engine Construction. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Virtual Event, USA) (ESEC/FSE 2020)*. Association for Computing Machinery, New York, NY, USA, 1140–1152. <https://doi.org/10.1145/3368089.3409710>
- [35] Manuel Rigger and Zhendong Su. 2020. Finding Bugs in Database Systems via Query Partitioning. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 211 (nov 2020), 30 pages. <https://doi.org/10.1145/3428279>
- [36] Manuel Rigger and Zhendong Su. 2020. Testing Database Engines via Pivoted Query Synthesis. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation (OSDI'20)*. USENIX Association, USA, Article 38, 16 pages.
- [37] Dušan Rychnovský and Holubová. 2015. Generating XML Data for XPath Queries. *Association for Computing Machinery*. (2015). <https://doi.org/10.1145/2695664.2695691>
- [38] Donald R. Slutz. 1998. Massive Stochastic Testing of SQL. In *Proceedings of the 24rd International Conference on Very Large Data Bases (VLDB '98)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 618–622.
- [39] Thodoris Sotiropoulos, Stefanos Chaliasos, Vaggelis Atlidakis, Dimitris Mitropoulos, and Diomidis Spinellis. 2021. Data-Oriented Differential Testing of Object-Relational Mapping Systems. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 1535–1547. <https://doi.org/10.1109/ICSE43902.2021.00137>
- [40] Ting Su, Yichen Yan, Jue Wang, Jingling Sun, Yiheng Xiong, Geguang Pu, Ke Wang, and Zhendong Su. 2021. Fully Automated Functional Fuzzing of Android Apps for Detecting Non-Crashing Logic Bugs. *Proc. ACM Program. Lang.* 5, OOPSLA, Article 156 (oct 2021), 31 pages. <https://doi.org/10.1145/3485533>
- [41] Milos Todic and Branislav Uzelac. 2012. Combined XML/XQuery generator. *Proceedings of the Fifth International Workshop on Testing Database Systems* (2012). <https://doi.org/10.1145/2304510.2304519>
- [42] Yuqing Wu, Namrata Lele, Rashmi Aroskar, Sharanya Chinnusamy, and Sofia Brenes. 2009. XQGen: An Algebra-Based XPath Query Generator for Micro-Benchmarking. In *Proceedings of the 18th ACM Conference on Information and Knowledge Management (Hong Kong, China) (CIKM '09)*. Association for Computing Machinery, New York, NY, USA, 2109–2110. <https://doi.org/10.1145/1645953.1646328>
- [43] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/1993498.1993532>
- [44] Hua Z, Lin W, Ren L, Li Z, Zhang L, Jiao W, and Xie T. 2023. GDsmith: Detecting bugs in Cypher graph database engines. *Proceedings of ACM SIGSOFT International Symposium on Software Testing and Analysis* (2023). <https://doi.org/10.48550/arXiv.2206.08530>
- [45] Qirun Zhang, Chengnian Sun, and Zhendong Su. 2017. Skeletal Program Enumeration for Rigorous Compiler Testing. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Barcelona, Spain) (PLDI 2017). Association for Computing Machinery, New York, NY, USA, 347–361. <https://doi.org/10.1145/3062341.3062379>
- [46] Yingying Zheng, Wensheng Dou, Yicheng Wang, Zheng Qin, Lei Tang, Yu Gao, Dong Wang, Wei Wang, and Jun Wei. 2022. Finding bugs in Gremlin-based graph database systems via randomized differential testing. *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis* (2022). <https://doi.org/10.1145/3533767.3534409>
- [47] Rui Zhong, Yongheng Chen, Hong Hu, Hangfan Zhang, Wenke Lee, and Dinghao Wu. 2020. Squirrel: Testing database management systems with language validity and coverage feedback. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 955–970.